

Lexicographic Reverse Search: Extreme-Point Enumeration of Local Credal Sets

IBM Research

Abstract

The credal-network family of inference engines in this library (CredalVE, IntervalBP, CredalCTE, ApproxLP, CredalIJGP) all consume the *extreme points* (vertices) of the local credal sets rather than the interval bounds directly. Converting a set of interval bounds $[\ell_i, u_i]$ on a conditional distribution into the vertex representation of the corresponding polytope is a *vertex-enumeration* problem, solved here by pyAgrum’s `intervalToCredal()` which runs the **Lexicographic Reverse Search (LRS)** algorithm of Avis and Fukuda. This note describes the geometry of the local credal sets, the LRS algorithm and its output-sensitive guarantees, works a complete example on the compiled credal network of `alarm.lcn`, and analyses why the enumeration — cheap for narrow families — becomes the dominant cost for wide or high-cardinality families.

1 From interval bounds to vertices

1.1 The local credal set of a family

After an LCN is compiled into a credal network (see `credal.network.py`), each node X with cardinality $d = |\mathcal{D}_X|$ and each configuration π of its parents carries a **local credal set**: a convex set of conditional distributions $P(X \mid \pi)$. In the interval representation produced by the compiler, this set is specified by lower/upper bounds $\ell_i \leq P(X=i \mid \pi) \leq u_i$ for each state $i \in \{1, \dots, d\}$. Geometrically it is the *interval-constrained probability simplex*

$$K(X \mid \pi) = \left\{ p \in \mathbb{R}^d : \sum_{i=1}^d p_i = 1, \quad \ell_i \leq p_i \leq u_i \quad (1 \leq i \leq d) \right\}. \quad (1)$$

This is a bounded polytope living in the $(d-1)$ -dimensional affine hyperplane $\sum_i p_i = 1$, cut out by the $2d$ interval facets. Downstream engines need its **V-representation**: the finite set $\text{ext } K(X \mid \pi)$ of vertices, each of which is a concrete probability distribution over X .

1.2 H- and V-representations

A convex polytope admits two dual descriptions:

- the **H-representation**: an intersection of half-spaces, $\{x : Ax \leq b\}$ (here, the $2d$ interval inequalities plus the normalization equality) — this is what the compiler stores;
- the **V-representation**: the convex hull of its vertices, $\text{conv}\{v_1, \dots, v_V\}$ — this is what the engines consume.

The **vertex-enumeration problem** is: given the H-representation, output every vertex. This is exactly what `gum.CredalNet.intervalToCredal()` does, once per (X, π) pair, in `CredalNetworkVertices._build`

```

self.credal_net = gum.CredalNet(self.bn_min, self.bn_max)
self.credal_net.intervalToCredal()    # LRS vertex enumeration

```

Note that the enumeration is run *per local credal set*, not on any global joint polytope: there is one polytope of the form (1) for every parent configuration of every node.

2 The LRS algorithm

2.1 Reverse search

Vertex enumeration can be done by pivoting (Avis–Fukuda’s *lrs*) or by the double description method (Motzkin’s *cdd*). pyAgrum uses **Lexicographic Reverse Search (LRS)** [1], a pivoting method with a decisive advantage: it uses memory independent of the output size and never stores the vertices it has already found.

Reverse search is a general technique for enumerating the nodes of an implicitly defined graph. For vertex enumeration the graph is the **skeleton** of the polytope — its vertices connected by edges. The ingredients are:

1. A *local search / pivot rule* f that, from any vertex, deterministically moves to a unique “parent” vertex. Under Bland’s/the lexicographic rule this is the reverse of the simplex pivot, and following f repeatedly from any vertex always terminates at the same unique *root* (the lexicographically-minimal optimal vertex of a chosen objective).
2. Because f is deterministic and acyclic, the “ f leads to parent” relation induces a **spanning tree** of the skeleton rooted at that vertex.
3. Reverse search enumerates the tree by DFS *from the root outward*: at each vertex it tries every adjacent edge (every reverse pivot); a neighbour is accepted as a child iff applying f to it returns to the current vertex.

The elegance is that the tree is never materialised. LRS holds only the current vertex and the current pivot indices — $O(d \cdot m)$ working memory — and reconstructs its path by pivoting, so it can enumerate astronomically many vertices in bounded space. The “lexicographic” qualifier is the perturbation rule that breaks ties consistently and thereby handles **degenerate** polytopes (more than d facets meeting at a vertex) — which the interval simplices (1) routinely are, since a vertex is pinned by choosing which of the $2d$ bounds are tight.

2.2 Output-sensitive complexity

Let a single local polytope (1) have

- dimension $d' = d - 1$ (states $d = |\mathcal{D}_X|$, minus the normalization);
- $m \approx 2d$ facets (the interval bounds);
- V output vertices.

LRS enumerates all V vertices in

$$O(d' \cdot m \cdot V) \text{ time,} \quad O(d' \cdot m) \text{ space.} \quad (2)$$

That is, the cost per vertex is *polynomial* ($O(d'm)$, one pivot table update) and the algorithm is **output-sensitive**: its running time is proportional to the size V of the answer it produces, with

Algorithm 1 Reverse search vertex enumeration (schematic)

```
1: procedure LRS( $A, b$ ) ▷ polytope  $\{x : Ax \leq b\}$ 
2:    $v \leftarrow$  root vertex (lex-min optimum of a generic objective  $c^\top x$ )
3:    $j \leftarrow 0$  ▷ index of the reverse pivot being tried
4:   repeat
5:     output  $v$  when first visited
6:     while  $j <$  number of candidate pivots at  $v$  do
7:        $j \leftarrow j + 1$ 
8:        $w \leftarrow \text{REVERSEPIVOT}(v, j)$ 
9:       if  $w$  is a vertex and  $f(w) = v$  then ▷  $w$  is a child
10:         $v \leftarrow w$ ;  $j \leftarrow 0$  ▷ descend
11:        break
12:      end if
13:    end while
14:    if no child was found then ▷ backtrack via the deterministic pivot  $f$ 
15:       $(v, j) \leftarrow f(v)$  and restore the pivot index at the parent
16:    end if
17:  until back at the root with all pivots exhausted
18: end procedure
```

polynomial overhead and no dependence of the *memory* on V . This is optimal in the sense that any algorithm must at least write its V outputs; LRS adds only a polynomial factor and constant space. (For non-degenerate input LRS is provably polynomial-total-time; the lexicographic rule extends this to the degenerate case at the cost of processing some “dictionaries” that do not emit a new vertex, which for the highly-degenerate interval simplices is a real but bounded overhead.)

3 Worked example: the credal network of alarm.lcn

3.1 The model

The LCN `examples/alarm.lcn` is

$$\begin{aligned} s_1 : 0.1 \leq P(B) \leq 0.2, & & s_2 : 0.05 \leq P(E) \leq 0.1, \\ s_3 : 0.6 \leq P(A \mid B \vee E) \leq 0.9, & & s_4 : 0.5 \leq P(\neg(C \oplus D) \mid A) \leq 0.7, \\ s_5 : 0.05 \leq P(A) \leq 0.1. & & \end{aligned}$$

Compiling it (`CredalNetwork.from_lcn`) yields a chain-graph credal network over **four nodes** $\{A, B, E, C-D\}$, where C and D are fused into a single compound node $C-D$ of cardinality 4 because the sentence s_4 couples them. The directed structure is $B \rightarrow A$, $E \rightarrow A$, and $A \rightarrow C-D$. The compiled interval local credal sets (one row per family interpretation) are:

node	(states, parents) config	ℓ	u
B	$B=0$	0.8000	0.9000
	$B=1$	0.1000	0.2000
E	$E=0$	0.9000	0.9500
	$E=1$	0.0500	0.1000
A	$A=0 \mid B=0, E=0$	0.9556	1.0000
	$A=1 \mid B=0, E=0$	0.0000	0.0444
	$\cdots \mid$ (other B, E configs)	0.0000	1.0000
$C-D$	$CD \mid A=0$	$u = (1, 0.7, 1, 0.5), \ell = 0$	
	$CD \mid A=1$	$u = (1, 0.5, 1, 0.7), \ell = 0$	

3.2 Enumerating each local credal set

LRS is run once per (node, parent-config) pair. The results (`cnv.extreme_points`) are:

Binary root nodes ($d = 2, d' = 1$). The polytope is a 1-dimensional segment; it has exactly 2 vertices, its endpoints.

$$K(B) = \text{conv}\{(0.8, 0.2), (0.9, 0.1)\},$$

$$K(E) = \text{conv}\{(0.9, 0.1), (0.95, 0.05)\}.$$

The binary child A ($d = 2$). Each parent configuration is again a segment with 2 vertices. For $B=0, E=0$ the interval $P(A=0) \in [0.9556, 1]$ gives

$$K(A \mid B=0, E=0) = \text{conv}\{(0.9556, 0.0444), (1.0, 0.0)\},$$

while the three configurations touching $B \vee E$ collapse to the whole edge $\text{conv}\{(0, 1), (1, 0)\}$ (the $[0, 1]$ interval is vacuous there).

The compound node $C-D$ ($d = 4, d' = 3$). This is where enumeration bites. The local set is the 3-dimensional interval simplex (1) with $m = 8$ facets.

- Under $A=0$ the upper bounds are $(1, 0.7, 1, 0.5)$; LRS returns the **4** unit vertices $\{(0, 0, 0, 1), (0, 0, 1, 0), (0, 1, 0, 0), (1, 0, 0, 0)\}$ (the upper bounds are loose enough that only the simplex corners survive).
- Under $A=1$ the upper bounds are $(1, 0.5, 1, 0.7)$; these cut the simplex into a polytope with **11** vertices, e.g.

$$(0.5, 0.5, 0, 0), (0.7, 0, 0.3, 0), (0.3, 0, 0, 0.7), (0, 0.5, 0.5, 0), (0, 0, 0.3, 0.7), \dots$$

each corresponding to a distinct choice of which interval bounds are active.

3.3 Why $C-D$ is the expensive family

The contrast is the whole point. The three binary nodes each yield 2 vertices per parent config in $O(1)$ work. The single 4-state node $C-D$ yields up to 11 vertices for *one* parent configuration — and it has 2 parent configs, so $4 + 11 = 15$ vertices come from this one family alone. The Upper Bound Theorem (next section) caps a $d'=3, m=8$ polytope at 12 vertices; the observed 11 is right at that ceiling. Fusing C and D into a cardinality-4 node — forced by the cross-atom sentence s_4 — is what pushed the enumeration from trivial to non-trivial.

4 Complexity for larger credal networks

4.1 Two sources of blow-up

Equation (2) is polynomial *in the output*, so the real question is how large the output V is, and how many polytopes must be enumerated. There are two independent exponential factors, both driven by the **family scope**.

(1) Vertices per polytope, exponential in cardinality d . By the **Upper Bound Theorem** (McMullen), a d' -dimensional polytope with m facets has at most

$$V_{\max}(d', m) = \binom{m - \lceil d'/2 \rceil}{m - d'} + \binom{m - \lceil (d'+1)/2 \rceil}{m - d'}$$

vertices. For a local interval simplex, $d' = d - 1$ and $m = 2d$, so V grows *super-polynomially in the node cardinality d* :

states d	polytope dim d'	facets m	max vertices $V_{\max}$
2	1	4	2
4	3	8	12
8	7	16	440
16	15	32	692,208

A node of cardinality $d = 2^k$ arises whenever k atoms are fused into one compound node (as $C, D \rightarrow C-D$ with $k = 2$, $d = 4$ above). Thus *each additional fused atom roughly squares the worst-case vertex count* of that family’s local polytope.

(2) Number of polytopes, exponential in parent scope. LRS is invoked once per parent configuration. A family whose parents have p atoms in total has 2^p parent configurations, hence 2^p separate polytopes to enumerate. The total enumeration cost of the whole network is therefore

$$\underbrace{\sum_{\text{node } X}}_{\text{linear in \#families}} \underbrace{\sum_{\pi \in \text{cfg}(\text{Pa}(X))}}_{2^{|\text{Pa atoms}|}} O(d'_X \cdot m_X \cdot V_{X,\pi}), \quad (3)$$

which is **linear in the number of families** but **exponential in each family’s scope** through both the 2^p configurations and the $V_{X,\pi}$ vertices per configuration (itself up to $\approx V_{\max}(2^c - 1, 2^{c+1})$ for a c -atom child).

4.2 The controlling parameter is treewidth, not n

Because the exponents in (3) are the child cardinality and the parent-scope, and both are bounded by the size of the largest family, LRS enumeration cost is governed by the **induced width** / **treewidth** of the moralized network, *not* by the raw number of variables n :

- **Chains, trees, polytrees** (small, fixed family scope): every family has $O(1)$ atoms, so each polytope is low-dimensional with $O(1)$ vertices, and the total cost is $O(n)$ — enumeration is essentially free.

- **k -trees** (bounded treewidth k): each family conditions on a k -clique, so a family spans $O(k)$ atoms, giving $O(2^k)$ parent configs and up to $V_{\max}(O(2^k))$ vertices each. Cost is *linear in n but exponential in k* — exactly why the `ktree/ktree-fr` benchmark generator parametrizes by treewidth k .
- **Dense / heavily-merged networks** (large families, e.g. under a large `merge_budget`, scheme D2): a merged super-family of flattened scope w has a child of cardinality up to 2^w and 2^w -ish parent configs. Both exponentials fire simultaneously and the enumeration dominates the entire pipeline — doubly exponential in w in the worst case.

4.3 Practical consequences in this codebase

- **The `enumerate_vertices=False` escape hatch.** CredalJT (scheme D5, `junction_nlp.py`) formulates its NLP directly from the interval bounds and the LMC equalities, so it *skips* LRS entirely (see `CredalNetworkVertices._build`). For wide families this avoids the exponential (3) altogether — the intervals are $O(2^{\text{scope}})$ to store but the vertex set they would generate is far larger.
- **Potential clustering.** CredalVE/CredalCTE offer `n_clusters` / `cluster_representative` knobs that *prune* the enumerated vertex set to a bounded number of representatives, trading tightness for a hard cap on the V that propagates downstream — a direct mitigation of factor (1).
- **The `.cn` cache pays this cost on every hit.** Loading a compiled `.cn` restores the interval bounds for free, but the engine still re-runs LRS (the intervals, not the vertices, are cached). Hence a cache hit’s reported build time is the stored `compile_time` *plus* local vertex enumeration — cheap for narrow families, non-trivial for wide ones. Caching the V-representation itself is possible but was rejected: it can be exponentially larger than the H-representation and would couple the `.cn` to pyAgrum.

5 Summary

LRS is an *output-sensitive, polynomial-per-vertex, constant-space* vertex-enumeration algorithm: enumerating a local credal set with V vertices costs $O(d'mV)$ time and $O(d'm)$ memory (Eq. (2)). The algorithm itself is not the bottleneck — the *number of vertices V* and the *number of local credal sets* are. Both grow exponentially in the **family scope** (Eq. (3)): the child cardinality inflates V super-polynomially (Upper Bound Theorem: $2 \rightarrow 2$, $4 \rightarrow 12$, $8 \rightarrow 440$, $16 \rightarrow \sim 700,000$ vertices), and the parent scope multiplies the number of polytopes by 2^p . Consequently LRS enumeration is linear in n but exponential in treewidth: free on chains and trees, tolerable on bounded-treewidth k -trees, and prohibitive on dense or heavily merged networks — for which the intended remedy is to skip enumeration (CredalJT) or to cap the vertex set (potential clustering).

References

- [1] D. Avis and K. Fukuda. *A pivoting algorithm for convex hulls and vertex enumeration of arrangements and polyhedra*. Discrete & Computational Geometry, 8:295–313, 1992.
- [2] D. Avis. *A revised implementation of the reverse search vertex enumeration algorithm*. In Polytopes—Combinatorics and Computation, pp. 177–198, Birkhäuser, 2000.